Assignment 1: Introduction to Operating Systems

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 14, 2026

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only.

1.1 Conceptual

Q1. A student claims that "the operating system is just the kernel, and everything else — the shell, the file managers, the compilers — is an ordinary application with no special status." Give a precise response that (a) distinguishes the *three* views of an operating system from the lecture, and (b) explains what, if anything, is special about the kernel relative to the shell. Your answer must name the hardware mechanism that actually makes the kernel special.

Q2. A runaway user process enters an infinite loop and never performs any I/O. Explain, step by step, why the machine does not hang forever. In your answer, name the two distinct hardware mechanisms involved (one that restricts *what* the process may do, and one that ensures the kernel regains control), and state what would go wrong if the second mechanism were removed.

1.2 Numerical

Q3. On a certain system, a system call costs a fixed 2 μs of kernel overhead, and an additional 0.5 μs whenever it passes its arguments through a parameter block (instead of registers). A copy utility copies a 4 MB file using a 4 KB user buffer, issuing `open` once, then one `read` and one `write` per buffer, then `close` once. The `open` and `close` calls use registers; every `read` and `write` uses a parameter block. Compute (a) the total number of system calls, (b) the total system-call overhead in milliseconds, and (c) the overhead as a percentage of the file's transfer time if the disk sustains 100 MB/s.

Q4. A time-sharing system programs a timer to interrupt the CPU every 10 ms. Servicing one timer interrupt (saving the context, running the scheduler, restoring a context) costs 12 μs on average. Compute the fraction of CPU time spent on timer handling, expressed as a percentage. Then state, in one sentence, how this overhead changes if the quantum is reduced to 1 ms, and why designers do not simply make the quantum as small as possible.

Concepts That Click

1.3 Design

Q5. You must add a new system call `spawn(path, argv, envp, flags, out_pid)` to a kernel. It takes a variable-length path string, a variable-length argument vector, a variable-length environment vector, an integer flags word, and returns a process id. Design the parameter-passing scheme you would use, choosing among registers, a parameter block, and the stack. Justify your choice by explaining why registers alone are insufficient here, and give the exact layout of the block you would pass.

Q6. A vendor is choosing between a *monolithic* kernel and a *microkernel* architecture for a device that must run unattended for years and whose file server occasionally contains latent bugs. (a) Describe what happens, in each architecture, when the file server crashes. (b) Recommend one architecture for this device and justify the recommendation using the trade-offs from the lecture. (c) State the single most important performance cost the recommended architecture pays, and one technique to mitigate it.

Concepts That Click